

과정 없는 결과의 가치

김현구, KAIST 수리과학과

AI는 최근 수학의 오랜 난제들을 연이어 풀어내고 있지만, 증명이 늘어나는 만큼 새로운 수학이 늘어나지는 않는다. 수학의 가치는 답 자체보다 답을 찾는 과정에서 쌓이는 이해에 있었기 때문이다. 프로그래밍도 같은 길을 걷고 있다. AI가 코드를 만들고 디버깅하고 유지 보수하는 일에서 사람보다 낫다고 인정하더라도, 현재의 코딩 에이전트(coding agent)는 결과만 남기고 사람이 이해를 쌓던 과정을 없앴다. 구현이 애초에 프로그래밍 가치의 작은 부분이었음을 인정하고, 왜 이 프로그램을 원하는가, 무엇을 얻는가, 누구에게 도움이 되는가를 묻는 데 집중해야 한다. 그리고 이 질문을 지키는 일은 개인이 아니라 공동체의 몫이다.

AI는 이제 인간 지성의 마지막 보루로 여겨지던 영역까지 넘보고 있다. 일상생활부터 직장까지 AI의 도움을 받지 않는 곳이 없고, 요즘은 AI를 신처럼 떠받드는 이야기를 어디서나 듣는다. 2026년 9월에는 OpenAI가 내부 모델로 밀레니엄 난제(Millennium Prize Problems) 중 하나인 나비에-스토크스 방정식의 해가 특정 조건에서 유한 시간 안에 폭발(blowup)함을 증명했다고 주장했다 [3]. 그 과정의 천박함에서 애써 눈을 돌린다면 이는 분명히 경이로운 업적이다. 그러나 내가 두려운 것은 AI의 능력 자체가 아니라, AI가 답을 대신 내주는 동안 우리가 답에 이르는 과정에서 얻던 이해를 잃는다는 점이다.

AI는 수학 분야에서 극적인 성과를 내고 있다. 불과 2년 전만 해도 AlphaProof는 사람이 형식 언어로 옮겨 준 국제수학올림피아드 문제를 푸는 데 최대 사흘이 걸렸다 [2]. 그러나 올해 OpenAI의 모델들은 단위 거리 추측(unit distance conjecture)의 반례를 찾았고, 이중 순환 덮개 추측(cycle double cover conjecture)의 증명과 삼각형의 다색 램지 수(multicolor Ramsey number)가 초지수적(super-exponential)으로 커진다는 증명을 발표했다. 단위 거리 추측의 반례를 검증한 수학자들은 그 논증이 이미 알려진 아이디어들에 결정적으로 기대고 있다고 밝혔다 [1]. 우리에게는 이미 모든 도구가 갖추어져 있었지만, 겨우 두 개의 손으로는 올바른 도구를 고르지 못했던 것이다. 반면 AI는 만 개의 손으로 만 개의 도구를 동시에 휘두를 수 있었다.

그러나 증명이 늘어난다고 수학이 발전하는 것은 아니다. 페르마의 마지막 정리는 아름다운 명제이지만, 그 자체로 중요한 명제는 아닐지도 모른다. 정리가 증명되었다는 사실보다 더 중요한 것은 그 과정에서 인류가 쌓아 올린 새로운 수학이며, 나아가 이를 엄밀화(formalize)하고 표준화(standardize)하여 이해하는 일이다. 미해결 문제는 이렇게 새로운 수학을 향한 길을 밝혀 주는 불빛이었다. AI가 기존 도구의 조합으로 문제를 풀어 버리면, 우리는 답을 얻는 대신 그 불빛을 잃는다. 그동안 수학계는 명제를 최초로 증명하는 일에 너무 큰 가치를 두었고, 이제 그 대가를 치르고 있다. 테렌스 타오(Terence Tao)가 AI 시대에 수학 연구의 목표와 가치가 무엇인지 되물은 것도 같은 맥락이다 [6]. 우리는 매일 새로운 증명을 얻고 있지만, 새로운 수학은 얻지 못하고 있다. 쌓여있는 증명의 산을 밟고 서서 우리가 곧 하늘에 가 닿으리라 말하지만, 그곳은 빛 한 점 없이 캄캄하다.

프로그래밍은 수학보다 먼저, 그리고 더 빠르게 같은 길을 걷고 있다. Anthropic의 Claude Code 책임자는 자신의 코드를 100% AI가 작성한다고 밝혔다 [4]. 과거의 도구들은 추상화 수준을 한 단계 올렸을 뿐 여전히 사람이 무언가를 써야 했던 반면, 지금의 AI는 사람의 말을 코드로 옮기는 과정 자체를 맡는다. 코드를 쓰는 과정은 이미 AI에게 넘어갔고, 우리는 이미, 혹은 가까운 미래에 AI가 충분히 좋은 코드를 만들어 내고 디버깅하고 유지 보수하는 일에서 사람을 뛰어넘을 것이라 가정해 보아야 한다. 그 뒤에 남는 것은 원하는 것을 정확히 기술하고, 만들어진 것이 정말 원하는 것인지 판단하는 더 어려운 일이다 [5]. 미래의 프로그래머는 코드를 쓰는 사람이 아니라 무엇을 만들지 정하고 만들어진 시스템을 이해하고 책임지는 사람이 될 것이며, 문제는 수학의 사례가 보여 주듯 과정을 AI에게 넘기는 순간 이해가 가장 먼저 사라진다는 데 있다.

현재 최고 수준의 코딩 에이전트가 가진 문제는 코드의 품질이 아니라, 결과만 남기고 과정을 없앤다는 데 있다. 마이어는 MIT 수업에서 AI로 대단한 코드를 만든 학생들이 정작 그 코드를 설명하지도 고치지도 못했다고 전한다 [5]. 현재 내 컴퓨터에도 내가 원하는 일을 정확하게 수행하지만 한 번도 읽어본 적 없는 코드가 여럿 있다. 그 코드들은 읽을 필요조차 없을 만큼 잘

동작하지만, 그것을 만드는 과정에서 나는 무엇을 얻었는가? 증명은 늘지만 새로운 수학은 늘지 않는 것과 같다. 여기서 잃는 것은 코드 한 줄 한 줄에 대한 이해가 아니라 무엇을 왜 만들었고 그 과정에서 무엇을 배웠는가에 대한 이해다. 과정이 사라지면 사람도 자라지 않는다. 미해결 문제가 새로운 수학을 낳았듯, 손으로 코딩하며 겪는 시행착오는 신입 개발자를 신뢰할 수 있는 전문가로 길러 왔다. 이 기초 경험 없이는 다음 세대의 숙련된 엔지니어를 길러 낼 방법이 없다 [4].

해결책은 구현이 프로그래밍 가치의 작은 부분이었음을 인정하고, 나머지에 집중하는 것이다. 사람이 거대한 프로그램을 만드는 것은 매우 어려운 일이었고, 우리는 이를 대단히 가치 있는 일로 여겨 왔다. 이제는 누구나 숙련된 프로그래머 100명분의 일을 순식간에 해낼 수 있다. 그렇다면 우리는 기존의 100배, 1000배의 가치를 만들어 내고 있는가? 그렇지 않다. 코딩은 원래 프로젝트 비용의 10~20%에 불과했으며 [5], 구현은 처음부터 가치의 작은 부분이었다. 나머지 큰 가치는 우리가 왜 이 프로그램을 원하는가, 그것을 만드는 과정에서 무엇을 얻는가, 그 결과로 사람들에게 어떤 도움이 되는가라는 자연스럽게 중대한 질문에 있다. 이 가치는 아직 요구공학(requirements engineering)과 확인(validation)의 형태로 사람에게 남아있다.

결국 미래의 프로그래밍에서 지켜야 할 것은 결과가 아니라 이해이며, 이는 개인이 아니라 공동체가 함께 깊어져야 할 몫이다. 수학 공동체와 프로그래머 공동체가 공유하는 가치가 있다: 지속 가능할 것, 다음 세대가 있을 것, 그리고 우리가 나아갈 방향을 고민할 것. 미래의 시니어 개발자 대신 연산 자원에 투자하는 지금의 방식은 지속될 수 없다 [4]. 신입 연구자와 개발자가 시행착오를 겪을 자리를 남겨 두지 않는 공동체에는 다음 세대가 없다. 미해결 문제가 수학이 나아갈 길을 밝혀 주었듯, 무엇을 왜 만들지 묻는 일이 프로그래밍이 나아갈 길을 밝힌다. 누가 먼저 답을 내는가를 겨루는 경쟁이 가려 놓은 이 가치들을 되찾아야 한다. AI가 키보드를 넘겨받은 시대에 프로그래머의 마지막 책임은 자신이 무엇을 왜 만들었는지 이해하는 것이다.

References

- [1] Noga Alon, Thomas F. Bloom, W. T. Gowers, Daniel Litt, Will Sawin, Arul Shankar, Jacob Tsimmerman, Victor Wang, and Melanie Matchett Wood. 2026. Remarks on the Disproof of the Unit Distance Conjecture. arXiv:2605.20695. Accessed: 2026-09-11. <https://arxiv.org/abs/2605.20695>
- [2] AlphaProof team and AlphaGeometry team. 2024. AI Achieves Silver-Medal Standard Solving International Mathematical Olympiad Problems. Google DeepMind. Accessed: 2026-09-09. <https://deepmind.google/blog/ai-solves-imo-problems-at-silver-medal-level/>
- [3] Joseph Howlett. 2026. AI May Have Just Solved a Million-Dollar Math Problem. The Field Will Never Be the Same. Scientific American. Accessed: 2026-09-11. <https://www.scientificamerican.com/article/ai-may-have-just-solved-a-million-dollar-math-problem-the-field-will-never-be-the-same/>
- [4] Logan Kugler. 2026. The End of the Coder? Communications of the ACM. Accessed: 2026-09-09. <https://cacm.acm.org/news/the-end-of-the-coder/>
- [5] Bertrand Meyer. 2026. Artificial Intelligence for Software Engineering: From Probable to Provable. *Commun. ACM* 69, 6 (2026), 46–49. doi:10.1145/3773295
- [6] Terence Tao. 2026. Mathematics in the Age of AI. Public lecture, International Congress of Mathematicians (ICM) 2026. Accessed: 2026-09-11. <https://www.youtube.com/watch?v=M0--ZH1lOzg>